Section A: Concept Application

Question 1: Explain how relational databases help maintain accuracy in expense records.

Answer: Relational databases maintain accuracy through **ACID properties** (Atomicity, Consistency, Isolation, Durability) and normalized table structures. Instead of repeating user names or category names across millions of rows—which risks typos and inconsistent data—the database stores data once in dedicated tables (users, categories) and links them using numeric keys. This eliminates data redundancy and guarantees that updates to a user's details instantly reflect across all their related financial records.

Question 2: Why are constraints important in personal finance data?

Answer:

Constraints serve as the fundamental data-quality guardrails for personal finance data:

- **PRIMARY KEY:** Prevents duplicate transactions by ensuring every single expense or user has a unique identifier.
- **FOREIGN KEY:** Prevents data fragmentation by ensuring an expense can never be assigned to a non-existent user or an undefined category.
- **NOT NULL / CHECK:** Prevents broken calculations by ensuring financial values like amount aren't left blank or registered as negative numbers where invalid.

Question 3: How does GROUP BY help analyze spending patterns?

Answer: The GROUP BY clause collapses individual transaction rows into distinct categorical buckets. When paired with aggregate functions like SUM(amount), AVG(amount), or COUNT(), it turns a chaotic list of daily receipts into actionable financial insights (e.g., showing you exactly how much money went to 'Rent' vs. 'Food' over a specific period).

Question 4: Explain a scenario where rollback is required during expense entry.

Answer: Imagine entering a split-bill transaction where money must be deducted from a bank balance table and added to an expenses table. If the system logs the expense row but the application crashes before updating the bank balance, the financial data becomes desynchronized. A ROLLBACK is triggered here to completely undo the partial entries, returning the database to its last clean state as if the broken transaction never happened.

Question 5: How do views help users track monthly expenses efficiently?

Answer: Views act as saved, pre-written queries that mask complex multi-table joins. Instead of typing out intricate INNER JOIN statements involving three tables every time a user checks their monthly dashboard, a view lets the application retrieve real-time aggregated data using a clean, simple SELECT * FROM ViewName statement.

Question 6: Why use triggers for automatic category or balance updates?

Answer:

Triggers eliminate manual calculations by running automatically behind the scenes in response to specified database events (INSERT, UPDATE, DELETE). For instance, whenever a new row is appended to the expenses table, an AFTER INSERT trigger can instantly recalculate and update a current_balance column in a separate user profile table without requiring extra code from the frontend application.

Section B: SQL Hands-On

Given Database Schema (DO NOT MODIFY)

```
users (
user_id INT PRIMARY KEY,
name VARCHAR(50),
email VARCHAR(100),
created_at DATE
);
categories (
category_id INT PRIMARY KEY,
category_name VARCHAR(50)
);
expenses (
expense_id INT PRIMARY KEY,
user_id INT,
category_id INT,
amount DECIMAL(10,2),
expense_date DATE,
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

1. DDL Understanding Without modifying the schema:

● Explain why foreign keys are used: Specifically, why must user_id in the expenses table correspond to a real ID in the users table?

A foreign key ensures **referential integrity**. It requires the `user_id` in the `expenses` table to match a real ID in the `users` table because an expense cannot exist without an owner—it prevents invalid or unassigned financial data from entering the system.

● Mention one issue that would occur if foreign keys were removed: Describe the risk of "Orphaned Records" (e.g., an expense existing for a user who has been deleted)

Without foreign keys, deleting a user from the `users` table leaves their corresponding rows in the `expenses` table untouched. These remaining rows become **orphaned records**—"ghost" data pointing to a non-existent user, which breaks application queries, skews financial analytics, and wastes storage.

2. DML Operations

Write SQL queries to:
● Insert 5 users into the users table with unique names and emails.
● Insert 3 categories (e.g., 'Food', 'Rent', 'Entertainment').
● Insert 10 expense records distributed across different users and categories.
● Update one incorrect expense: Use the UPDATE command to change the amount of a specific expense_id.
● Delete one expense: Remove a record where the amount is less than a specific value

Step 1: Insert 5 Users

This query adds 5 unique users with distinct IDs, names, emails, and registration dates.

```
INSERT INTO users (user_id, name, email, created_at) VALUES
(1, 'Alice Smith', 'alice@email.com', '2026-01-15'),
(2, 'Bob Jones', 'bob@email.com', '2026-01-20'),
(3, 'Charlie Brown', 'charlie@email.com', '2026-02-01'),
(4, 'Diana Prince', 'diana@email.com', '2026-02-10'),
(5, 'Evan Wright', 'evan@email.com', '2026-02-15');
```

Step 2: Insert 3 Categories

This query establishes the main budget categories that expenses will map to.

```
INSERT INTO categories (category_id, category_name) VALUES
(1, 'Food'),
(2, 'Rent'),
(3, 'Entertainment');
```

Step 3: Insert 10 Expense Records

This query distributes 10 distinct purchases across the 5 users and 3 categories created in the previous steps.

```
INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date) VALUES
(101, 1, 1, 15.50, '2026-03-01'),
(102, 1, 2, 1200.00, '2026-03-01'),
```

(103, 2, 1, 42.75, '2026-03-02'),
(104, 2, 3, 25.00, '2026-03-03'),
(105, 3, 2, 1100.00, '2026-03-01'),
(106, 3, 1, 8.50, '2026-03-04'),
(107, 4, 3, 60.00, '2026-03-05'),
(108, 4, 1, 12.30, '2026-03-06'),
(109, 5, 2, 1350.00, '2026-03-01'),
(110, 5, 3, 18.00, '2026-03-07');

Step 4: Update One Incorrect Expense

This query targets a specific record (`expense_id = 101`) and updates its `amount` column to correct a entry error.

```
UPDATE expenses
SET amount = 18.25
WHERE expense_id = 101;
```

Step 5: Delete One Expense by Value Condition

This query removes a record (`expense_id = 106`) only because its price falls below the specified threshold of $10.00.

```
DELETE FROM expenses
WHERE expense_id = 106 AND amount < 10.00;
```

3. Data Retrieval

Write queries to
● Display all expenses with details: Show the expense_date, amount, name (from users), and category_name (from categories) using INNER JOIN.
● Show total expense amount per category: Use SUM(amount) and GROUP BY category_name.
● Display users sorted by total spending: List user names and their total sum of expenses, ordered from highest to lowest.

1. Display All Expenses with Details

This query uses `INNER JOIN` to fetch related data from the `users` and `categories` tables, replacing raw foreign IDs with readable names.

```
SELECT
    e.expense_date,
    e.amount,
    u.name,
    c.category_name
FROM expenses e
INNER JOIN users u ON e.user_id = u.user_id
INNER JOIN categories c ON e.category_id = c.category_id;
```

Expected Output:

expense_date	amount	name	category_name
2026-03-01	18.25	Alice Smith	Food
2026-03-01	1200.00	Alice Smith	Rent
2026-03-02	42.75	Bob Jones	Food
2026-03-03	25.00	Bob Jones	Entertainment
2026-03-01	1100.00	Charlie Brown	Rent
2026-03-05	60.00	Diana Prince	Entertainment
2026-03-06	12.30	Diana Prince	Food

2026-03-01	1350.00	Evan Wright	Rent
2026-03-07	18.00	Evan Wright	Entertainment

(Note: The $8.50 food expense for Charlie Brown is absent here because it was deleted in Step 5 of the previous question).

2. Show Total Expense Amount per Category

This query uses `SUM` and `GROUP BY` to aggregate individual transactions into a single total per budget category.

```
SELECT
    c.category_name,
    SUM(e.amount) AS total_expense_amount
FROM expenses e
INNER JOIN categories c ON e.category_id = c.category_id
GROUP BY c.category_name;
```

Expected Output:

category_name	total_expense_amount
Food	73.30
Rent	3650.00
Entertainment	103.00

3. Display Users Sorted by Total Spending

This query groups transactions by user and calculates their total spending, using `ORDER BY DESC` to rank them from highest to lowest spender.

```
SELECT
    u.name,
    SUM(e.amount) AS total_spending
FROM users u
INNER JOIN expenses e ON u.user_id = e.user_id
GROUP BY u.user_id, u.name
ORDER BY total_spending DESC;
```

Expected Output:

name	total_spending
Evan Wright	1368.00
Alice Smith	1218.25
Charlie Brown	1100.00
Bob Jones	67.75
Diana Prince	72.30

4. Views

● Create a view named ActiveUsersView: This view should list the name and email of any user who has logged more than 5 individual expense records.
● Query the view: Write a simple SELECT statement to show all data from ActiveUsersView.

1. Create the View (`ActiveUsersView`)

This query creates a virtual table using `COUNT(e.expense_id)` combined with a `HAVING` clause to filter only for users who have logged more than 5 individual expenses.

```
CREATE VIEW ActiveUsersView AS
SELECT
   u.name,
   u.email
FROM users u
INNER JOIN expenses e ON u.user_id = e.user_id
GROUP BY u.user_id, u.name, u.email
HAVING COUNT(e.expense_id) > 5;
```

2. Query the View

You can query this view just like a standard table to see the results.

```
SELECT * FROM ActiveUsersView;
```

Expected Output
Based on the sample data inserted in Question 2 (where each user only has 2 expenses), this query will currently return an empty set:

text
Empty set (0.00 sec)

(Note: If a user like 'Alice Smith' logs 4 more expenses in the database, her name and email will automatically appear here when you run this `SELECT` statement).